SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	Shahana	**Reg. No.:**	192524479
Section / Batch:	Slot A	**Date:**	26/08/2026

Code of Conduct

I Shahana (192524479)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shahana_____

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:	**100 Marks**	

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Comparative Analysis

Q1. Linear Probing vs Separate Chaining

Aspect	Linear Probing	Separate Chaining
a) Clustering effect	Suffers from primary clustering; consecutive occupied slots form growing clusters, worsening probe sequences.	No clustering; each key hashes to its own bucket/chain independently.
b) Memory usage	Fixed-size array, no pointer overhead, but wastes space when sparse (load factor must stay < 1).	Extra memory for pointers/node overhead, but handles load factor > 1 gracefully.
c) Performance under high load factor	Degrades sharply as load factor approaches 1; time grows roughly as $O(1/(1-\alpha))$.	Degrades gracefully; average chain length = α, so performance decreases linearly.
d) Suitability for large-scale applications	Better for cache-sensitive, memory-constrained systems (good cache locality), but needs careful resizing to keep α low.	Generally more suitable for large-scale/unpredictable-size applications — tolerates high load factors without catastrophic slowdown.

Q2. Quick Sort vs Merge Sort — Sorting a 10-Million-Node Linked List

Aspect	Quick Sort	Merge Sort
Memory (in-place vs extra space)	In-place on arrays (O(log n) stack), but loses this advantage on linked lists since pivot partitioning needs random access.	Naturally suited to linked lists — merging done via pointer manipulation, no extra array space needed (O(log n) recursion stack only).
Cache behavior	Poor on linked lists — random jumps across nodes during partitioning, no cache locality.	Sequential access pattern during merging — better cache behavior even on linked lists.
Time complexity	Average O(n log n), worst case $O(n^2)$ (bad pivot choices, e.g., sorted data).	Consistent O(n log n) in best, average, and worst cases.

Preferable for linked lists: Merge Sort — linked lists lack O(1) random access, so Quick Sort's partitioning becomes inefficient, while Merge Sort's sequential merge process works naturally with pointers and needs no extra space.

For arrays: Quick Sort is often preferred — arrays allow O(1) random access, so in-place partitioning is fast and cache-friendly, and its average-case constants beat Merge Sort, which needs O(n) extra space.

Q3. Quadratic Probing vs Double Hashing

Aspect	Quadratic Probing	Double Hashing
a) Clustering issues	Avoids primary clustering but suffers from secondary clustering — same initial slot means identical probe sequence.	Minimal clustering — probe sequence depends on a second hash function, so keys take different paths.
b) Collision resolution efficiency	More efficient than linear probing, but may fail to find an empty slot unless table size is prime and load factor ≤ 0.5.	More efficient and thorough — closest to ideal uniform probing, distributes keys better.
c) Suitability for dense hash tables	Works fine for moderate load factors but becomes unreliable as the table fills up.	More suitable for dense (high load factor) tables — distributes probes more uniformly.

Q4. Best-Case vs Worst-Case Time Complexity of Sorting Algorithms

Aspect	Best-Case Complexity	Worst-Case Complexity
a) Practical significance	Shows potential efficiency on favorable/nearly-sorted input (e.g., Insertion Sort's O(n) best case).	Gives a guaranteed upper bound, critical for real-time/safety-critical systems needing predictable performance.
b) Impact on algorithm selection	If input is often nearly sorted, algorithms with good best-case (Insertion/Bubble Sort) are attractive.	If input is unpredictable/adversarial, algorithms with strong worst-case guarantees (Merge Sort, Heap Sort) are safer than Quick Sort ($O(n^2)$ worst case).

Q5. Recursive vs Iterative Implementations of Quick Sort

Aspect	Recursive Implementation	Iterative Implementation
a) Space complexity (stack usage)	Uses call stack; O(log n) average but O(n) worst case (unbalanced partitions) due to call frames.	Uses an explicit stack data structure; same worst case O(n), but can be capped at O(log n) by pushing the larger partition first.
b) Ease of implementation	Simpler and more intuitive — mirrors divide-and-conquer logic directly, less code.	More complex — requires manually managing a stack to store sub-array boundaries, more error-prone.
c) Risk of stack overflow	Higher risk — deep recursion on large/skewed inputs can exhaust the call stack.	Lower risk — explicit stack is typically heap-allocated, not bound by call-stack size limits.